Ders 12 Genişletilmiş Notu: İnsan Hataları ve COCOMO II Makaleleri ile İlk Uygulama Sunumları

Ders 12 Genişletilmiş Notu: İnsan Hataları ve COCOMO II Makaleleri ile İlk Uygulama Sunumları

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dersin Akışı
- 2. 1. Sunum: Yazılımcıların Beceri Tabanlı Hataları
- 3. 2. Sunum: DO-178C Süreçlerinde COCOMO II ile Efor Tahmini
- 4. 3. Sunum: Scrum'ın Yazılım Proje Yönetimine Etkisi
- 5. 1. Uygulama: Sera Sıcaklık Kontrol Sistemi
- 6. 2. Uygulama: Robot Kontrol Sistemi
- 7. 3. Uygulama: Stok Yönetim Sistemi
- 8. Şelale Modelinde İş Adımları Arasındaki İlişkiler

1. Dersin Akışı

Bu derste konu anlatımı yapılmadı. Ders öğrenci sunumlarına ayrıldı. Önce kalan üç makale sunumu dinlendi, ardından üç uygulama projesi sunuldu. Hoca sunumların arasına kısa açıklamalar ve geri bildirimler ekledi.

Makale sunumlarının üçü de dersin farklı bir başlığına bağlanıyor. Birincisi yazılım proje yönetiminin insan (people) boyutuna, ikincisi maliyet ve efor tahminine, üçüncüsü çevik yöntemlere değiniyor. Uygulama sunumlarında ise derste görülen proje yönetim belgelerinin (risk tablosu, iş kırılım yapısı, Gantt şeması, kalite planı, maliyet tahmini) küçük bir proje üzerinde nasıl hazırlandığı görülüyor. Dersin en önemli vurgusu 3. uygulamanın sonunda geldi: Şelale modelinde alt iş adımlarının hepsi bitiş-başlangıç ilişkisiyle bağlanmak zorunda değildir. Bu konu Bölüm 8'de ayrıca ele alınıyor.

2. 1. Sunum: Yazılımcıların Beceri Tabanlı Hataları

2.1 Konu ve Dersle İlişkisi

Makale, yazılım geliştiricilerin kod yazarken yaptığı hataları nasıl azaltabileceğini inceliyor. IEEE Transactions on Software Engineering dergisinde yayımlanmış. Sunumda yayın yılı önce 2020 olarak söylendi, sonra 2021 olabileceği belirtildi.

Hoca konunun yazılım kalite dersiyle kesiştiğini söyledi. Yine de bu derse uygun bulmasının nedenini arkadaşlarına da açıkladı: Hata burada teknik açıdan değil, yazılımcının bakış açısından ele alınıyor. Yani konu, yazılım proje yönetimindeki insan (people), yani iş gücü başlığına bağlanıyor.

2.2 Problem ve Önceki Çalışmalar

Yazılımcılar kod yazarken hata yapmaya yatkındır. Şimdiye kadar hataların çoğunlukla teknik nedenlerine bakılmış ve çözüm olarak karmaşık araçlar önerilmiş. Hataları insan açısından

ayrıntılı değerlendiren çalışma ise çok az. Bu makale insan hatası teorisini kullanıyor ve yazılımcılarla mülakat yaparak iki soruya cevap arıyor:

- Profesyonel yazılımcılar yazılım geliştirirken hangi beceri tabanlı hataları yapar?
- Yazılımcılar bu hataları nasıl azaltabilir?

Sunum, yazılım tarihinden üç olayla başladı. Ulaşım, bankacılık, iş uygulamaları gibi yazılımın olduğu her alanda hata görülebiliyor ve bu kusurların temeline inildiğinde genellikle insan hatası bulunuyor.

- **Kripto para borsası:** Bir borsada yazılım hatası yüzünden 850 bin bitcoin kaybedildiği anlatıldı. Sunan öğrenci, borsada tutulan paraların başka insanlara da ait olduğunu hatırlattı.
- **Heartbleed:** Yaygın kullanılan açık kaynak kriptografi kütüphanesi OpenSSL'de ortaya çıkan bir yazılım hatası. Kütüphane yaygın olduğu için çok sayıda sistem etkilendi.
- **Mars Climate Orbiter:** Ölçü birimi uyuşmazlığı yüzünden Mars'a gönderilen uydunun parçalanması. Sunumda bu olay, yazılım hatasından kaynaklanan en pahalı felaketlerden biri olarak geçti.

  Ders sonrası not: Altyazıda "Mutibox" ve 2011 diye geçen olay büyük olasılıkla Mt. Gox borsasıdır. Borsa yaklaşık 850 bin bitcoin kaybını büyük olasılıkla 2014'te açıklayarak çökmüştür; 2011 büyük olasılıkla saldırıların başladığı döneme denk gelir.

  Ders sonrası not: Sunumda Mars Climate Orbiter için "Avrupa ve Amerika'da kullanılan metrik sistemler farklıydı" denildi. Doğrusu büyük olasılıkla şudur: İki ekip de ABD'deydi (Lockheed Martin ve NASA JPL). Birinin İngiliz, diğerinin metrik birim kullanması uyuşmazlığa yol açtı.

İnsan hatası teorisi özellikle tıp ve sağlık alanında hataları analiz etmek ve azaltma stratejileri geliştirmek için kullanılmış. Yazılım alanında insani durumların kod yazmaya etkisi de araştırılmış. Ancak çoğu çalışma, geliştiricilerin depoya (repository) gönderdiği kusurlu kodları inceleyerek hatalara dolaylı olarak bakmış. Depo madenciliği (repository mining), duygu analizi ve doğal dil işleme gibi tekniklerle geliştiricinin ruh hâlinin kod düzeltmelerine etkisi araştırılmış. Örneğin üzgün yazılımcıların kusurları düzeltmesinin mutlu yazılımcılara göre daha uzun sürdüğü gösterilmiş.

2.3 Hataların Nedenleri

Sunumda yazılım hatalarının kaynağında yazılımcının olduğu, hataların çoğunun da geliştirme sırasında ortaya çıktığı söylendi. Nedenler üç grupta toplandı:

- **Psikolojik nedenler:** En sık görülen gruptur. Kod yazarken bir arkadaşın işi bölmesi, çevre etkileriyle konsantre olamamak, öfke ve hayal kırıklığı gibi olumsuz duygular bu gruba girer.
- **Fiziksel nedenler:** Yorgunluk ve uykusuzluk.
- **Çevresel nedenler:** Sıcaklık ve soğuk gibi etkenler.

2.4 İnsan Hatası Teorisi

Çalışma, Rasmussen'in çerçevesini kullanıyor. Bu çerçevede hatalar üç başlıkta toplanır:

- **Beceri tabanlı (skill-based):** İyi anlaşılmış görevlerle ilişkilidir. Yeni bilgi kullanmadan yapılan otomatik davranışlardaki hatalardır.
- **Kural tabanlı (rule-based):** Kayıtlı kurallarla kontrol edilen, tanınabilir hata kalıplarıdır.
- **Bilgi tabanlı (knowledge-based):** Analitik düşünme ve kayıtlı bilgi gerektiren, beklenmedik ve bilinmeyen durumlardaki hatalardır. Bilinçli akıl yürütme içerir.

Makale kural ve bilgi tabanlı hatalar üzerinde neredeyse hiç durmuyor, beceri tabanlı hatalara odaklanıyor. Gerekçe, bu hataların fark edilmesinin ve düzeltilmesinin daha kolay olmasıdır.

Hatalar bir de yürütme (execution) ve planlama (planning) hataları olarak ikiye ayrılıyor. Yürütme hataları beceri tabanlıdır. Planlama sırasında olan hatalar ise yanılgıdır (mistake) ve kural ya da bilgi tabanlıdır. Sunumda yanılgının biraz acemiliğe dayalı hatalar gibi tanımlandığı söylendi. Yürütme hataları da ikiye ayrılıyor:

- **Kayma (slip):** Dikkatsizlik sonucu oluşur. Örnek: klavyede yanlış tuşa basmak.
- **Unutma (lapse):** Yapılmak istenen şeyin yapılmasının istemeden unutulmasıdır, yani hafıza hatasıdır. Görevli işi yapmak ister ama unutur.

  Ders sonrası not: Altyazıda Rasmussen'in "fiziksel hataları" üç başlıkta topladığı geçiyor; bağlama göre doğrusu "insan hataları"dır. Yürütme ve planlama ayrımı "Erasmus'a göre" diye geçiyor. "Yine" dendiğine göre sunan öğrenci Rasmussen'i kastetmiş olabilir, ama kayma, unutma ve yanılgı sınıflaması büyük olasılıkla James Reason'a aittir. Altyazıdaki "Ziple Lep" de büyük olasılıkla "slip ve lapse" ifadesidir.

2.5 Sekiz Beceri Tabanlı Hata Türü

Bu sekiz tür mülakatın temelidir. Her biri önce tanımlanmış, sonra gerçek dünya örneğiyle açıklanmış. Sunumdaki örnekler günlük hayattandır:

- **İhmal (omission):** Süreçteki önemli bir adımın atlanması. Örnek: çay yaparken çaydanlığın altını yakmayı unutmak.
- **Tekrar (repetition):** Tamamlanmış bir adımın yeniden yapılması. Örnek: kaynamış suyun altını yeniden yakmak.
- **Tersine çevirme (reversal):** İş dizisinin dışından bir şeyle ilgilendikten sonra dizide yanlış işe dönmek. Örnek: amaç ayakkabıları çıkarıp terlik giymek. O sırada ceket düşüyor, ceket kaldırılıyor ve terlik yerine çıkarılmış ayakkabılar yeniden giyiliyor.
- **Bölünme sonrası ihmal:** Dış bir neden yüzünden bir şeyin unutulması. Örnek: dışarı çıkmak için ceketi almak, telefon çalınca cevap vermek ve ceketsiz çıkmak.
- **Çifte yakalama kayması (double capture slip):** Sunan öğrenci bunu Türkçeye tam çeviremediğini söyledi. İlişkili başka bir işin de istemeden yapılmasıdır. Örnek: amaç ayakkabıyı çıkarmakken çorabı da çıkarmak.
- **Azaltılmış niyet (reduced intentionality):** Gecikmeler yapılacak işe müdahale eder ve iş unutulur. Örnek: fırını açıp içindekilere bakmaya başlamak, sonra ne istendiğini hatırlayamamak.
- **Algısal karışıklık (perceptual confusion):** Tekrarlanan görevler otomatikleşir, girdiler kabaca kabul edilir ve bu da karışıklığa yol açar. Örnek: dolaptan hep süt şişesi alan biri, başka bir şişe istendiğinde alışkanlıkla yanlış şişeyi alır.
- **Girişim hatası (interference error):** İki aktif plan ya da aynı planın farklı bölümleri birbirine dolanır. Örnek: telefon konuşmasını bitirirken içeri girenlere selam vermek isterken "alo, kimsiniz" demek.

2.6 Yöntem

Çalışmaya, tamamı endüstride çalışan 27 yazılımcı katılmış. Katılımcılar bir Twitter duyurusu ve kulaktan kulağa yayılma yoluyla bulunmuş. Mülakatlar Skype üzerinden ya da yüz yüze yapılmış ve 30-45 dakika sürmüş. Akış şöyledir:

- Sekiz hata türü tanımlanır ve gerçek dünya örnekleriyle açıklanır.
- Yazılımcıya kod yazarken bu hataları yapıp yapmadığı sorulur.
- Yapıyorsa, azaltmak için hangi stratejiyi uyguladığı sorulur.
- Görüşmeler kaydedilir ve görüşmeciler tarafından ayrı ayrı yazıya geçirilir.
- Sekiz türün dışında bildirilen hatalar da kategorilere ayrılır. Görüşmecilerin ayrı ayrı çıkardığı hatalar ve stratejiler karşılaştırılıp yeniden kategorize edilir.

Sonuçta 57 farklı hata teması ortaya çıkmış. Bunlar sekiz türün dışında, yazılımcıların kendi anlatımlarından çıkan temalardır. Neden, hata ve sonuç olarak gruplanmış ve her birinin ne sıklıkla anıldığı bir tabloda verilmiş.

2.7 Bulgular

Yazılımcılar hatanın kendisinden çok neden ve sonuçlarına odaklanmış. Sekiz tür arasında en çok ihmal ve bölünme sonrası ihmal dile getirilmiş. Katılımcıların çoğu hataları iki nedene bağlamış: geliştirme ortamının karmaşıklığı ile konsantrasyon ve odaklanma sorunları.

Geliştirme ortamının karmaşıklığı. Birden fazla dil, araç ve bağımlılık bilgisi ile bunlar arasındaki bağlantıların yönetilmesi karmaşıklığı artırıyor. Karmaşık bir görev zaten başlı ba-

şına bir yükken, karmaşık bir ortamda yapılması yükü daha da artırıyor. Yazılımcıların verdiği örnek, yazılan bir betiğin hedef veritabanı yerine başka bir veritabanında çalıştırılmasıdır.

Konsantrasyon bozukluğu. Nedenleri arasında görev değişimi, dikkat dağınıklığı ve iş baskısı öne çıkıyor. Bunlar yazılımcının görev sırasındaki farkındalığını azaltıyor. Örnekler: bir e-postaya cevap verdikten sonra göreve dönünce ne yapıldığının unutulması; mesai bitimine yakın, yarına yetişmesi gereken bir işin verilmesi.

Hata azaltma stratejileri dört yüksek seviyeli başlıkta toplanmış: geliştirici, süreç, araç ve yönetim.

- **Geliştirici stratejileri:** En sık önerilen gruptur. Yazılımcılar kodlama alışkanlıklarını değiştirirken iradelerini kullandıklarını, kendi hakimiyetlerini artırarak hataları azaltabileceklerini düşünüyor. Bölünmelere karşı kulaklık kullanıyor ve bildirimleri kapatıyorlar. Ancak planlı destek olmadan irade ve disiplini artırmak zorlaşıyor; irade tek başına her zaman işe yaramıyor.
- **Süreç stratejileri:** Kontrol listesi, kod gözden geçirme ve test. Bunlar kodlama sırasında hatanın oluşmasına pek engel olmuyor, daha çok yapılan hatanın görülmesini ve hızlı düzeltilmesini sağlıyor.
- **Araç stratejileri:** Yazılımcılar otomasyon araçlarının hataları azaltmaya yardım ettiğini söylüyor. Özellikle git ve benzeri programlar anılmış.
- **Yönetim stratejileri:** Çok az yer alıyor. Yine de yönetimin iş yüklerini iyi dağıtması, ödüllendirmesi ve eğitim vermesi gerektiği belirtiliyor.

Sonuçların iki yönü ilginç bulunmuş:

- Bu çalışmaya kadar yazılımcıların süreç ve otomasyon araçlarına daha çok eğilimli olduğu düşünülüyordu. Oysa en çok yazılımcı temelli stratejiler önerildi. Yazılımcılar kod yazarken oluşan hatalardan kendilerini sorumlu tutuyor.
- Ara vermek ve dinlenmek hata azaltma stratejisi olarak pek anılmamış. Sunumdaki yorum şudur: Yorgunluk kolay fark edilir ve çözümü basittir. Yorgunsan biraz dinlenir, sonra devam edersin.

Stratejiler dahili (psikolojik, yazılımcıya bağlı) ve harici (araç, süreç, yönetim) olarak da ayrılmış. Harici stratejiler daha çok kusurların etkisini tespit etmeye yönelik. Yazılımcıların dahili stratejileri uygulamasına da yardım ediyor gibi görünüyor. Örneğin kontrol listesi böyle bir destek sağlıyor.

2.8 İsviçre Peyniri Modeli

Sunumda başarısızlığa karşı savunma oluşturan bir yaklaşım olarak İsviçre peyniri modeli (Swiss cheese model) gösterildi. Model insan merkezli değil, sistem merkezlidir. Hatanın baştan sona ilerleyip sistem başarısızlığına dönüşmesini önlemeye odaklanır. Bir yazılımcı hata yapmış olsun. Pull request ve kod gözden geçirme (code review) gibi savunma katmanları sayesinde bu hata canlıya çıkmadan durdurulabilir. Sunumda bu yöntemin yazılımda henüz çok kullanılmadığı, daha da geliştirilmesi gerektiği söylendi.

> **Ek bilgi (derste anlatılmadı):** Modelin adı bir benzetmeden gelir. Her savunma katmanı delikleri olan bir peynir dilimi gibi düşünülür. Bir hata ancak tüm dilimlerin delikleri aynı hizaya geldiğinde sisteme kadar ulaşır. Model büyük olasılıkla James Reason'a aittir.

2.9 Öneriler

- **Durum farkındalığını (situational awareness) geliştirmek:** Durum farkındalığı, bir görevi yaparken çevrede olup bitenden haberdar olmak ve sonraki adımı tahmin edebilmektir. Birçok hatanın nedeni bunun kaybedilmesidir. Örneğin küçük sanılan bir problem, derine inildikçe büyür ve yazılımcıyı asıl görevden uzaklaştırır. Yazılımcılar buna karşı iradeye dayalı stratejiler geliştirmiş, ama yalnız irade başarıyı sınırlıyor. Sunumda destek yaklaşımları geliştirilmesi gerektiği söylendi. Sunan öğrenci makalenin bu konuda çok belirli bir çözüm sunmadığını da ekledi.

- **Bilişsel yetenekleri geliştirmek:** Odaklanma, hatırlama, disiplin ve dikkat, hataların çoğuyla ilişkili. Bu yetenekler farkındalığı da etkiliyor. Çalışma ortamı ve iş baskısı bunları olumsuz etkiliyor. Çeşitli eğitimlerle geliştirilebileceği öneriliyor.
- **Kontrol listesi kullanmak:** Her şeyi hatırlamaya çalışmak yerine görevleri işaretlemek, yazılımcıyı hatırlama sorumluluğundan kurtarıyor.
- **Araçlardan yararlanmak:** Kod çakışmaları araçlarla iyi yönetilebiliyor.
- **Hızlı geri bildirim döngüleri kurmak:** Eşli programlamada (pair programming) birinin dikkati dağılsa bile diğeri odaklanmış kalabiliyor. Pull request'te başka bir yazılımcı kodu gözden geçirip hatayı bildiriyor, böylece hata daha hızlı düzeltiliyor.

Sunumda yazılımcılardan alıntılar da gösterildi. Durum farkındalığı için şu ifade okundu: "Kodun ne için yazıldığını anlamak, nasıl yazıldığını anlamaktan daha önemlidir."

> **Ders sonrası not:** Altyazıda bu yetenekler için "bilgisayar yetenekleri" ve "bilimsel yetenekler" geçiyor. Bağlama göre doğrusu "bilişsel (cognitive) yetenekler"dir.

2.10 Sonuç ve Hocanın Geri Bildirimi

Makale, insan hatası teorisini kullanarak iki sorusuna cevap veriyor: Yazılımcıların yaptığı beceri tabanlı hataları ve bunları azaltma yollarını ortaya koyuyor. Karmaşık geliştirme ortamının başlıca hata nedeni olması ilginç bir sonuç olarak sunuldu. Stratejilerin kategorize edilmesi de konunun anlaşılmasına katkı sağlıyor.

Hoca üç noktayı belirtti:

- Bildiri değil dergi makalesi seçilmesi ve makalenin IEEE Transactions on Software Engineering'den güncel bir makale olması güzel.
- Konu iyi, ama sunum süresi iyi kullanılamadı. Hoca sunum sırasında slayt sayısını sorup hızlanılmasını istemişti.
- Çalışma geliştirilebilir. İş hayatında olan öğrenciler benzer bir anket yapıp bu çalışmayı hata üzerinden genişletebilir.

Hoca kendisinin de daha önce bir yüksek lisans ve iki bitirme projesi öğrencisiyle insan faktörü üzerine anket çalışmaları yaptığını söyledi. Yüksek lisans çalışmasında insan faktörü açıklanmış, başka bir kesimde anket düzenlenmiş ve sonuçlar irdelenmişti.

3. 2. Sunum: DO-178C Süreçlerinde COCOMO II ile Efor Tahmini

3.1 Konu ve Amaç

Makale 2019 yılında Brezilya'da, Journal of Aerospace Technology and Management dergisinde yayımlanmış. DO-178C süreçlerini izleyen, güvenlik kritik (safety-critical) bir yazılım projesinde gereken eforun COCOMO II ile tahmin edilmesini anlatıyor. Vaka çalışmasının amacı, yazılım efor tahmini araştırmalarını desteklemek ve uygulayıcılara gerçek veri sağlamaktır. Havacılıkta projeler çok uzun sürdüğü için böyle bir yönteme ihtiyaç var.

Sunumda şu gözlem aktarıldı: Geçmişte bir uçağın geliştirilmesinde en büyük zorluk aerodinamikti, şimdi yazılımdır. Örnek olarak Boeing 787 Dreamliner verildi: 14 milyon satırlık kod ve yaklaşık 8 yıllık geliştirme süresi.

Efor tahmini, istenen sonuca ulaşmak için gereken maliyet, zaman ve eforu öngörmek için önemlidir. Sunumda gecikme ve kayıpları açıklamak için de kullanıldığı söylendi.

3.2 Efor Tahmin Yöntemleri

Teknikler iki sınıfa ayrılıyor:

- **Algoritmik olmayan yöntemler:** Uzman ve deneyimli kişilerin karşılaştırmalarına, çıkarımlarına, analojilerine ve fikirlerine dayanır. Matematiksel denklem kullanılmaz.
- **Algoritmik yöntemler:** Ölçütler matematiksel denklemlerle eforla ilişkilendirilir. Genelde kod satır sayısı referans alınır ve model bunu efor tahminine dönüştürür. COCOMO II bu gruptadır.

3.3 COCOMO II

COCOMO, Constructive Cost Model (yapısal maliyet modeli) ifadesinin kısaltmasıdır. İlk sürümü 1981'de ortaya çıkmış, 2000'de geliştirilmiştir. Efor kişi-ay (person-month, PM) cinsinden hesaplanır. Sunumda denklemlerin girdileri olarak bir sabit, bin satır cinsinden kod boyutu, efor çarpanları (EM) ve ölçek faktörleri (SF) sayıldı. Ayrıca geliştirme süresi (Time to Develop, TDEV) de hesaplanıyor.

> **Ek bilgi (derste anlatılmadı):** COCOMO II denklemleri büyük olasılıkla slaytta şu biçimdedir: $PM = A \cdot Size^{E} \cdot \prod_i EM_i$ ve $E = B + 0{,}01 \sum_j SF_j$. Süre de $TDEV = C \cdot PM^{F}$ biçiminde hesaplanır. Burada A, B, C model sabitleridir ve F üssü E'ye bağlıdır.

Beş ölçek faktörü çok düşükten çok yükseğe altı seviyede değerlendiriliyor:

- **PREC:** Önceki projelerle benzerlik.
- **FLEX:** Gereksinim ve zamanlama açısından geliştirme sürecinin esnekliği.
- **RESL:** Risk değerlendirme analizleri sonucunda elde edilen parametre.
- **TEAM:** Ekibin entegrasyon seviyesi ve insan faktörleri.
- **PMAT:** Sürecin CMMI'ya göre olgunluk seviyesi.

> **Ders sonrası not:** Sunumda altı seviyenin "çok düşükten çok yükseğe" olduğu söylendi. Bu aralıkta beş ad vardır (çok düşük, düşük, nominal, yüksek, çok yüksek). Altıncı seviye büyük olasılıkla "ekstra yüksek"tir (extra high).

On yedi çarpımsal maliyet faktörü dört grupta toplanıyor: ürün (5), donanım (3), insan (6) ve proje (3). Sunumda vaka için seçilen değerler de okundu. Bunları Bölüm 3.5'teki tabloda birlikte veriyoruz.

3.4 Güvenlik Kritik Yazılım ve DO-178C

Bir yazılım, insan güvenliği açısından tehlikeli veya kritik bir donanım ve yazılımı kontrol ediyorsa güvenlik kritik yazılımdır. Bu yazılımlar genellikle gömülü ve gerçek zamanlıdır. Tek bir arıza felakete yol açabilir. Belli standartlar izlenerek geliştirilirler:

- Havacılıkta DO-178C.
- Uzayda NASA-STD-8719.13C.
- Tıpta ISO/IEC 62304.

DO-178C, havacılıkta ekipman tasarımında geliştiriciler, kurulumcular ve kullanıcılar için standart belirleyen bir rehberdir. Rehber olmasına rağmen, havacılık otoriteleri asgari operasyonel performans spesifikasyonu belgelerinde ona atıf yapar ve bu süreçlerin izlenmesini ister. Otoriteler güvenlik kritik yazılımlarda DO-178C uygunluğu talep eder.

Standart, arızaların mürettebat veya uçak üzerindeki etkisini beş seviyede ele alıyor. Olasılıklar uçuş saati başınadır ve hata oluşma olasılığının bu değerlerin altında olması gerekir. Yazılım seviyesi arttıkça gerçekleştirilmesi gereken hedef (objective) sayısı da artar.

Seviye	Etki	Olasılık (uçuş saati başına)	Hedef sayısı (sunum)
A	Felaket (catastrophic), uçağın kırımına götürebilir	10^{-9}	71
B	Tehlikeli (hazardous)	10^{-7}	69
C	Büyük (major)	10^{-5}	62
D	Küçük (minor)	10^{-3}	24
E	Güvenlik etkisi yok	yok	yok

> **Ders sonrası not:** Sunumda D seviyesi için 24 hedef söylendi; DO-178C'deki değer büyük olasılıkla 26'dır.

Seviye, sistem güvenliği değerlendirme süreciyle belirlenir. Burada arızalar, işlev kayıpları ve bunların etkileri incelenir. Bunun için ayrı bir standart olan ARP 4761 kullanılır. Hata ağacı analizleri ve fonksiyonel tehlike analizleriyle uçağın ve ekipmanın fonksiyonları değerlendirilir.

Soru: Hoca sunumu durdurup DO'nun açılımını sordu. **Cevap:** Sunan öğrenci "Design Organization" olarak bildiğini söyledi. DOD'un ABD Savunma Bakanlığı (Department of Defense) standartları olduğunu ekledi. DO standartlarının ABD tarafından çıkarıldığını, Avrupa'daki karşılıklarının ED belgeleri olduğunu anlattı. Otoriteler de farklıdır: ABD'de FAA, Avrupa'da EASA. İkisinin de yaklaşık aynı içerikte rehber dokümanları var.

> **Ders sonrası not:** Sunumda DO "Design Organization" olarak açıklandı; bu doğru değildir. DO büyük olasılıkla standardı yayımlayan RTCA'nın belge numarası önekidir (RTCA DO-178C). Avrupa karşılığı büyük olasılıkla EUROCAE ED-12C'dir.

3.5 Vaka Çalışması

Proje, Brezilya'da 20 yıldır uçuş kontrol sistemleri üzerine yazılım geliştiren bir şirkete ait. Şirket, uçuş kontrolünün yazılımla yönetildiği fly-by-wire teknolojisi üzerinde çalışıyor. Yazılım A seviyesindedir. Proje Temmuz 2013'te başlamış, Aralık 2017'de bitmiştir. Tam zamanlı olarak aynı anda en fazla 48 yazılım mühendisi çalışmış. Bir alt yüklenici basit donanım arayüzleri ile dijital girdi ve çıktıları yönetmiş. Şirket ise kontrol teorisi, test fonksiyonları ve hata yönetimi gibi alanlarda tasarım yapmış.

Kod boyutu uzman görüşü ve benzetmeyle 140 bin satır olarak tahmin edilmiş. Ölçek faktörleri şöyle seçilmiş: PREC nominal, FLEX çok düşük, RESL nominal, TEAM çok düşük, PMAT çok yüksek. Bu değerlerle hesaplanan E üssü 1'den büyük çıkmış. Sunumda bunun anlamı şöyle açıklandı: Proje boyutu iki katına çıktığında sarf edilen efor iki katından fazla olur.

Efor çarpanları için seçilen değerler:

Grup	Çarpan	Seçilen değer
Ürün	Yazılım güvenilirliği	Çok yüksek
Ürün	Veritabanı boyutu	Nominal
Ürün	Ürün karmaşıklığı	Çok yüksek
Ürün	Yeniden kullanılabilirlik	Nominal
Ürün	Dokümantasyon ihtiyacı	Çok yüksek
Donanım	Yürütme süresi kısıtı	Yüksek
Donanım	Depolama kısıtı	Nominal
Donanım	Platform oynaklığı	Nominal
İnsan	Analist yeteneği	Düşük
İnsan	Programcı yeteneği	Çok yüksek
İnsan	Personel sürekliliği	Çok yüksek
İnsan	Analist deneyimi	Düşük
İnsan	Programcı deneyimi	Nominal
İnsan	Dil ve araç deneyimi	Nominal
Proje	Yazılım araçlarının kullanımı	Çok düşük
Proje	Çok konumlu geliştirme (multi-site development)	Çok düşük
Proje	Geliştirme takvimi	Düşük

Proje V modeliyle yürütülmüş. Hoca, öğrencilerin V modelini yazılım mühendisliği dersinden bildiğini söyleyerek ayrıntıya girilmesini istemedi. Sunumda COCOMO II'nin varsaydığı şelale (waterfall) ile DO-178C süreçleri karşılaştırıldı: DO-178C'de entegrasyon ve doğrulama süreçlerinin ağırlığı daha fazla.

3.6 Tahmin ile Gerçekleşenin Karşılaştırılması

Tahmin, 1 ay = 152 saat alınarak 1257 kişi-ay çıkmış. Plan, 41 tam zamanlı personelle 30,6 ayda 140 bin satır kod yazmaktı. Tutarlılık kontrolü (ara adım eklendi):

$$\frac{1257 \text{ kişi-ay}}{41 \text{ kişi}} \approx 30{,}66 \text{ ay}$$

Bu değer sunumdaki 30,6 ile uyumludur.

Gerçekleşen değerler ise şöyle:

Ölçüt	Tahmin	Gerçekleşen
Efor (kişi-ay)	1257	1780
Süre (ay)	30,6	54
Ortalama personel	41	yaklaşık 32,9
Kod boyutu (satır)	140.000	170.000

Gerçekleşen personel için de kontrol yapılabilir (ara adım eklendi): $1780/54 \approx 32{,}96$. Temmuz 2013-Aralık 2017 aralığı da iki uç ay dahil 54 aydır.

Aşamalara göre personel sayıları şöyle okundu: planlama 7, gereksinim yönetimi 12, tasarım 40, kodlama 45, entegrasyon 50, doğrulama 55.

> **Ders sonrası not:** Sunumda aynı anda en fazla 48 yazılım mühendisi çalıştığı söylendi, ama entegrasyon için 50, doğrulama için 55 kişi okundu. Bu iki bilgi çelişiyor. Altyazıdan çelişkinin nedeni anlaşılmıyor.

Modelin doğruluğu göreli hata büyüklüğüyle (Magnitude of Relative Error, MRE) ölçülmüş. Sunumdaki tanım, sarf edilen ve tahmin edilen efor farkının sarf edilen efora oranıdır. Sunumda sonuç 0,4 yani %40 hata olarak verildi. Bu tanımla verilen sayılar kullanılırsa:

$$MRE = \frac{|1780 - 1257|}{1780} = \frac{523}{1780} \approx 0{,}29$$

> **Ders sonrası not:** Sunumda MRE 0,4 denildi; verilen tanım ve sayılarla sonuç yaklaşık 0,29'dur (%29). Farkı tahmine bölersek $523/1257 \approx 0{,}42$ çıkar. Sunumdaki değer büyük olasılıkla bu hesaptan gelir.

3.7 Sonuç ve Hocanın Geri Bildirimi

Sunumdaki sonuç şudur: COCOMO II, DO-178C süreçlerini izleyen güvenlik kritik bir yazılımda, özellikle 71 hedefin tamamlanmasını bekleyen A seviyesinde yetersiz kalıyor. Gerekçeler şunlardır:

- Gereksinimlerin gözden geçirilmesi, kod incelemesi, test sonuçlarının gözden geçirilmesi ve testler son derece zaman alıcıdır.
- Bazı faaliyetlerde kişilerin bağımsız olması gerekir. Örneğin doğrulama (verification) ve geçerli kılma (validation) faaliyetlerini geliştiriciden başkası yapmalıdır.

Hoca, derste görülen COCOMO'yu somut bir uygulamada gösteren bir makale seçildiğini söyledi. Sürenin de iyi kullanıldığını belirtti.

4. 3. Sunum: Scrum'ın Yazılım Proje Yönetimine Etkisi

4.1 Konu ve Amaç

Makalenin adı "Influence of Agile Methodology on Software Project Management" olarak okundu. IEEE'de 2019'da yayımlanmış, sunan öğrenci konferans olarak hatırladığını söyledi. Makale, Scrum'ın yazılım projeleri üzerindeki etkisini anketle ölçüyor. Sunan öğrenci anlatımı 3-4 destekleyici makaleyle güçlendirdiğini de söyledi.

4.2 Arka Plan

Yazılımlar hayatın her alanına girdi, ama birçok proje başarısızlıkla sonuçlanıyor. Nedenleri arasında planlama eksikliği ile proje yönetimi tekniklerinin ve araçlarının eksikliği var. Sonuçlar maddi ve manevi olabiliyor. Üç örnek verildi:

- Kanser tedavisinde kullanılmak üzere geliştirilen bir yazılımdaki hatalar yüzünden hastalar ölmüş.
- Bir uçuşun teknik özelliklerindeki bir sorun yüzünden başarısız olması; maddi kaybı 8,5 milyar dolar olarak verildi.
- Almanya'da bir banka kartı hatası yüzünden kullanıcılar kartlarını kullanamamış; zarar 6 ile 10 milyon dolar arasında.

  Ders sonrası not: Altyazıda "Airline 5" diye geçen olay büyük olasılıkla Ariane 5 roketinin 1996'daki ilk uçuşunda yazılım hatası yüzünden kaybıdır. Kanser tedavisi örneği de büyük olasılıkla Therac-25 cihazıdır.

Makale, yazılım proje yönetiminin 10 bilgi alanını temel alıyor: entegrasyon, kapsam, zaman, maliyet, kalite, insan kaynakları, iletişim, risk, tedarik ve paydaş yönetimi. Başarılı bir proje iş gereksinimlerini karşılamalı, zamanında ve bütçe dahilinde teslim edilmeli, gereken destek sağlanmalı ve beklenen işi yapmalıdır.

Çevik metodoloji, dinamik geliştirme ortamlarına ayak uydurmak için yinelemeli bir yaklaşım sunuyor. Geleneksel yöntemlerin hızlı değişimlere cevap vermedeki başarısızlığını gideriyor. Gereksinim değişikliklerine karşı maliyet, kapsam ve kaliteyi karşılayabilecek esnekliğe sahip. Başlıca amaçlarından biri riski azaltmaktır. Bunu projeyi küçük parçalara ayırarak yapar.

Scrum, en çok tercih edilen çevik modellerden biridir. Sunuma göre sektörün %65'i Scrum'a geçmiş. Üç rol var: ürün sahibi, geliştirme ekibi, Scrum Master. Aşamaları başlatma, planlama, uygulama, kontrol etme ve yayınlamadır. İşler 3-4 haftada tamamlanabilecek sprintlere bölünüyor ve her parçaya bir proje gibi yaklaşılıyor. İzleme günlük ve haftalık toplantılarla yapılıyor. Hoca Scrum'ın bilindiğini söyleyerek bu kısmın hızlı geçilmesini istedi.

4.3 Anket ve Sonuçları

Anket 30'dan fazla yazılım şirketiyle yapılmış ve 10 bilgi alanının bir kısmına dair sorular sorulmuş. Yanıtlar 1-5 arası katılım ölçeğiyle derecelendirilmiş.

- **Kapsam yönetimi:** Kapsamı tanımlamak ve yönetmek zordur, müşteriyle görüşmeler sonucunda sürekli değişebilir. Müşteriyle sürekli toplantı yapıldığı için Scrum bu değişikliklere hızlı çözüm getiriyor. Etki olumlu bulunmuş.
- **Zaman ve maliyet yönetimi:** Scrum'ın maliyete olumlu etkisi %95,2 olarak verildi. Sprintler ve toplantılar sayesinde zaman kolay takip ediliyor, riskler azalıyor.
- **Kalite yönetimi:** Zamanında, maliyete uygun, kapsamı karşılayan ve beklendiği gibi davranan ürün kaliteli sayılıyor. Yanıtların %15'i kesinlikle katılıyorum, %55'i katılıyorum. Toplam %70 olumlu (ara adım eklendi).
- **Risk yönetimi:** Günlük ve haftalık toplantılar ile sprintler riski azaltıyor. Yanıtların yaklaşık %14'ü olumlu, %66'sı "her zaman bir risk vardır" görüşünde.
- **İnsan kaynakları yönetimi:** Scrum ekip çalışmasını ve iş bitirenin diğerlerine yardım etmesini getiriyor. Ekibe uygun kişi katılması da gerekiyor. Yanıtlarda %25 ve %50 değerleri olumlu etki olarak okundu.

4.4 Sonuç

Sunumdaki sonuç şudur: Scrum, proje süresi, maliyet, kapsam, kalite ve risk üzerinde olumlu etkili. Riski azaltıyor, maliyeti kontrol ediyor, kaliteli ürünün zamanında tamamlanmasına yardım ediyor. Hoca bu sunum için ayrıca bir değerlendirme yapmadı, yalnız makalenin yayın yerini sorup kaynakların gösterilmesini istedi.

5. 1. Uygulama: Sera Sıcaklık Kontrol Sistemi

5.1 Konu ve Amaç

İki öğrenci, basit bir sera sıcaklık kontrol sisteminin proje planını sundu. Amaç, seralardaki koşulları daha pratik hale getirmek için sıcaklığı Android uygulama üzerinden izlemek ve kontrol etmektir. Sunanlar, proje planını görmek için aktif olarak daha çok Android tarafını geliştirdiklerini söyledi.

5.2 Sistem ve Kullanım Senaryoları

Sistemin bileşenleri şunlardır: Arduino Uno, LM35 sıcaklık sensörü, internete çıkmak için Ethernet Shield, MySQL veritabanı, prototip için fan ve ısıtıcı, sıcaklığı göstermek için 16x2 LCD ekran, bir IoT platformu ve Android uygulama.

> **Ders sonrası not:** Altyazıda "Ubitox" diye geçen IoT platformu büyük olasılıkla Ubidots'tur.

Kullanım senaryoları (use case) şunlardır:

- Kullanıcı girişi ve profil güncelleme.
- Sera yönetimi: fan ve ısıtıcıyı açıp kapatma.
- Anlık sıcaklığı, ölçüldüğü saatle birlikte görüntüleme.
- Saatlik verilerden günlük rapor görüntüleme (çubuk ve çizgi grafik).

5.3 Risk Analizi

Riskler bir tabloda toplanmış. Sütunlar riskin etkisi, olasılığı, tipi, B planı ve C planıdır. Risk tipleri Z (zaman), K (kullanıcı), T (teknik) ve P (personel) olarak kodlanmış.

Risk	Tip	Etki ve olasılık	B planı	C planı
Eğitim ve deneyim eksikliği	Altyazıda net değil	Yüksek etki, düşük olasılık	Proje ile ilgili eğitim	Proje süresini uzatma
Arayüzün verimli kullanılamaması	Kullanıcı	Yüksek etki, orta olasılık	Arayüzü kullanıcı dostu olacak şekilde düzeltme	Arayüz çatısını değiştirme
Arduino ve Ethernet Shield'ın internete bağlanamaması	Teknik	Yüksek etki, orta olasılık	Yedek malzeme, uzman desteği	Cihaz ya da marka değiştirme
Veritabanının Arduino'ya bağlanamaması	Teknik	Yüksek etki, orta olasılık	Yedek malzeme	Cihaz, marka ya da veritabanı sistemi değiştirme
Sensör ölçümlerinin yanlış olması	Teknik	Altyazıda net değil	Yedek sensör	Farklı sensör kullanma
Ekranın doğru çalışmaması, kullanıcının yanlış yönlendirilmesi	Teknik ve kullanıcı	Altyazıda net değil	Uzman desteği, yedek ekran	Cihaz ya da marka değiştirme
Pandemi nedeniyle personelin hastalanması	Personel ve zaman	Altyazıda net değil	Bilgi yedekliliği	Proje süresinin uzaması

Pandemi riski için Scrum'ın bir avantajı anıldı: Tüm ekip yapılan işten haberdar olduğu için bir kişi hastalanınca diğerleri işe devam edebilir.

5.4 Zaman Planı

Proje Scrum ile planlanmış: product owner, geliştirme ekibi ve Scrum Master rolleri tanımlanmış. Dört sprint var, her biri iki hafta. Sunanlar Scrum'da sprintlerin iki üç hafta sürdüğünü, bu yüzden iki hafta seçtiklerini söyledi. Her sprint test aşamasıyla bitiyor ve ortaya çıkan prototip test ediliyor.

- **Sprint 1:** IoT sisteminin kurulması, ana devrenin Arduino'ya bağlanması, Arduino Uno programlaması, sensör bağlantısı, sıcaklık değişimini izlemek için LED'lerin bağlanması ve aralıklarının ayarlanması, test.
- **Sprint 2:** Android Studio ile arayüz geliştirme, arka plandaki operasyonların arayüze atanması (paralel, iki geliştirici), test.
- **Sprint 3:** Ethernet Shield'ın bağlanması, veritabanı-Arduino bağlantısı ve veritabanı-Android bağlantısı (paralel), test.
- **Sprint 4:** Isıtma ve soğutmanın devreye entegrasyonu, Android'de rapor görüntüleme (paralel), test.

Plan, iş kırılım yapısı (Work Breakdown Structure, WBS) ve Gantt şemasıyla gösterilmiş. İki görev arasında bitiş-başlangıç ilişkisi örneği verildi: Arduino bağlantısının bittiği gün Uno programlaması başlıyor. Plandaki "eksi bir gün" gecikmesi, sonraki görevin öncekinin bittiği gün başlaması anlamına geliyor. Paralel yürüyen işler de şemada görülüyor.

> **Ders sonrası not:** Sunumda sprintler arasında "start-to-finish" ilişkisi olduğu söylendi. Bir sprint bitince diğerinin başladığı anlatıldığına göre doğrusu finish-to-start'tır.

5.5 Kalite Planı ve Hocanın Geri Bildirimi

Dört kalite ölçüsü belirlenmiş:

- **Kapsama uygunluk:** SRS'te belirtilen kapsamla karşılaştırma.
- **Zaman planına uygunluk:** Zaman çizelgesi ve Gantt şemasının dışına çıkılmaması.
- **Bütçeye uygunluk:** Proje plan dokümanındaki bütçe durumuna göre.
- **Geliştirme modeline uygunluk:** Belirlenen modele sadık kalınması. Bunu product owner inceliyor.

Hoca, seçilen konuda bir proje planının özeti gibi hazırlandığını söyledi.

6. 2. Uygulama: Robot Kontrol Sistemi

6.1 Konu ve Kapsam

İki öğrenci, çevreden görüntü ve video kaydı alan bir robotun uzaktan yönetildiği sistemi sundu. Robot Bluetooth ile bağlanıyor. Kullanıcılar mobil ve masaüstü uygulamalardan robotu kontrol ediyor, çevreyi izliyor ve kayıt alıyor.

Varsayımlar ve kısıtlar şunlardır:

- Robot 50 cm uzunluğunda ve yürüyebiliyor. Kol uzunluğu 20 cm, bacak ve gövde uzunluğu 25 cm.
- Kayıt e-posta adresi, Facebook ya da Twitter hesabıyla yapılıyor ve adres doğrulanıyor.
- Bluetooth modülünün kapasitesi nedeniyle kontrol menzili 25 metre.
- İleri, geri, sağ, sol olmak üzere dört yönde hareket ve gerektiğinde zıplama.
- En fazla 100 MB video, 720p kalitesinde video ve fotoğraf.
- Son bir aylık kullanım verileri (aktiflik süreleri gibi) görüntüleniyor.

6.2 Yöntem, Ekip ve Bütçe

Süreç çevik ve Scrum ile yürütülüyor. Her sprint 21 gün, yani 3 hafta. Her sprintin sonunda tüm ekip ve yöneticilerin katıldığı değerlendirme toplantısı yapılıyor.

Bütçe üç kalemden oluşuyor: donanım (robot prototipi, iletişim modülü, bilgisayar, entegre devreler, kablolar) 100.000 TL, personel 588.000 TL, pazarlama 200.000 TL. Toplam 888.000 TL (ara adım eklendi).

Ekip yapısı:

Ekip	Kıdemli	Yeni başlayan
Analiz	1	2
Backend	1	2
Frontend	1	1
Haberleşme	1	1
Test	1	0

Bunlara bir de ürün sahibi ekleniyor. Toplam 12 kişi (ara adım eklendi).

Zaman planı MS Project ile yapılmış. Üç seviyeli bir WBS kullanılmış. Sunumda örnek olarak planlama ve analiz aşamaları gösterildi, gerçekleme, tasarım ve bakım dokümanda yer alıyor. Sunanlar projenin her aşamasında tüm ekip üyelerinin sprint koştuğunu söyledi.

6.3 Riskler

Riskler altı başlıkta toplanmış:

Risk	Tip	Önlem
Proje planında gecikme	Altyazıda net değil	Testleri daha yoğun ve hızlı yapmak, plan hazırlanırken olası gecikmeleri hesaba katmak
Geliştirme ortamlarının kurulamaması	Altyazıda net değil	Kurulum desteği almak, ortamı firma bünyesine almak
Ekipten birinin ayrılması	Personel	Görüş ve sıkıntılarla ilgilenerek memnuniyeti artırmak
Ekip yetkinliğinin yetersiz olması	Personel	Eğitim, takım içi dayanışma ve yardımlaşma
Karışık, yorum satırı içermeyen, anlaşılırlığı düşük kod	Teknik	Kodlama standartları, eşli programlama
Yetersiz test	Ürüne doğrudan etki eder	Teste yeterli efor ve süre ayırmak

6.4 Sistem Özellikleri ve Gereksinimler

Arayüz üç ekrandan oluşuyor:

- **Giriş ve kayıt ekranı:** E-posta, kullanıcı adı ve şifre ile kayıt, e-posta ve şifre ile giriş.
- **Kontrol ekranı:** Dört yön oku ve zıplama butonu, robotun kamera görüntüsü, video kaydını başlatıp bitiren ikon, anlık fotoğraf için kamera ikonu. Kayıtlar kullanıcının cihazına kaydediliyor. Zıplama, önde engel varken kullanılıyor.
- **Bilgi ve rapor ekranı:** Pil, bakım ve sinyal bilgileri yüzde olarak gösteriliyor. Son bir ayın aktiflik süreleri bir diyagramla veriliyor. Kullanıcı buradan pilin bittiğini ya da bakım gerektiğini anlayabiliyor.

Fonksiyonel olmayan gereksinimler şunlardır:

- Kolay anlaşılır ekranlar, göz yormayan renkler, okunaklı yazı boyutu ve font.
- Doğru ve anlaşılır bilgi, hata ve uyarı mesajları. Kullanıcı mesajı okuyunca ne yapacağını anlamalı.

- Telefon, bilgisayar ve tablette çalışma.
- Düşük yanıt süresi.
- E-posta ve şifre gibi kişisel verilerin güvenli saklanması.

Kullanılacak araçlar Jira, NetBeans, Oracle SQL Developer ve Trello olarak sayıldı.

6.5 Hocanın Geri Bildirimi

Hoca, robot kontrol sistemi için derste belirtilen proje yönetim belgelerinin hazırlandığını söyledi.

7. 3. Uygulama: Stok Yönetim Sistemi

7.1 Konu ve Kapsam

Bir öğrenci, internette hazır bulduğu bir stok yönetim yazılımı için proje planı ve testler hazırladı. Uygulama küçük ve orta büyüklükteki işletmeler için. Ürün ekleme, güncelleme ve silme, kategori yönetimi, kullanıcı yönetimi ve raporlar içeriyor. Sunumda amaç, işletmenin elindeki fiziksel malların gelecekteki gereksinimleri karşılayacak biçimde etkin yönetilmesi olarak açıklandı.

Soru: Sunan öğrenci, önceki derste birim test (unit test) yazılmasının da istendiğini duyduğunu söyledi ve bu yüzden üç modül için birim test yazdığını anlattı. **Cevap:** Hoca önce çevik yapılıp yapılmadığını, testlerin başta mı oluşturulduğunu sordu. Sonra birim testin yazılım mühendisliği ve yazılım kalite derslerinin konusu olduğunu söyledi. Yine de fazladan yapılmasında sorun görmedi ve önce sistemin diğer belgelerinin anlatılmasını istedi.

7.2 Yöntem ve Plan

Şelale modeli kullanılmış. Akış şöyledir ve her belgede müşteri revizyonu alınıp onay beklenir:

```
Başlama planı -> SRS -> SPMP -> Yazılım tasarım dokümanı
    -> Kodlama -> Test -> Teslim
```

Testlerde sorun çıkmazsa teslimat yapılıyor. Projede 5 kişi çalışıyor: proje yöneticisi, grafik tasarımcı, backend geliştirici, tasarımları HTML'e çeviren frontend geliştirici ve test uzmanı.

Plan ve kontrol belgeleri şunlardır:

- **Çalışma aktiviteleri:** İş kırılım yapısı ProjectLibre ile hazırlanmış. Müşteri toplantıları ile belge teslimi ve revizyonları kilometre taşı olarak işaretlenmiş.
- **Proje kontrol planı:** Takvim sabit. Tüm aşamalar ProjectLibre'de tanımlanıyor ve ekip bu plana göre hareket ediyor.
- **Kalite kontrol planı:** Her kilometre taşında yapılan işlemler testten geçiriliyor ve müşteri onayı alınıyor.
- **Risk yönetim planı:** Risk, işe etkisi ve olasılığı tabloda. Önlemler raporda ayrıntılı veriliyor.
- **Kabul planı:** Belgelerin belirlenen tarihlerde ve IEEE standartlarına uygun teslim edilmesi, ürünün dokümantasyonda belirtilen sonucu vermesi.

Kullanılan araçlar: ProjectLibre, GIMP, PHP, HTML5, CSS, JavaScript, MySQL, LibreOffice ve veritabanı tasarımı için DataGrip. Sunan öğrenci Linux kullandığı için MS Project yerine ProjectLibre'yi seçtiğini, GIMP'i de Photoshop'un alternatifi olarak kullandığını söyledi.

> **Ders sonrası not:** Altyazıda "IT-FIL standartları" ve "DataGrid" geçiyor. Doğrusu büyük olasılıkla IEEE standartları ve JetBrains'in DataGrip aracıdır.

7.3 Maliyet Tahmini

Tahmin üç adımda yapılmış:

- **Fonksiyon noktası (function point):** Yazılıma giriş sayısı, kullanıcıların aldığı çıktı sayısı, sorgulama sayısı, kütük sayısı ve dış ara birim sayısı çıkarılmış ve toplam puan hesaplanmış.

- **Değer ayarlama soruları:** Ders slaytlarındaki “güvenli yedekleme ve geri yükleme gerekli mi?” gibi sorular cevaplanmış ve buradan bir puan elde edilmiş.
- **Efor ve maliyet:** Sonuçlar PHP’nin katsayısıyla çarpılarak kod boyutuna dönüştürülmüş, COCOMO ile hesap yapılmış.

Sonuç yaklaşık 7,8 ay ve 30 bin dolar olarak verildi. Birim maliyet “kişi başı 10 dolar” olarak söylendi.

7.4 Test Planı ve Birim Testler

Test planında kullanıcı girişi, kullanıcı kaydı, ürün ekleme ve kategori ekleme için senaryolar yazılmış. Örneğin giriş senaryosunda e-posta ve şifre ile giriş yapılıyor, hatalı girişte kullanıcı bekletiliyor. Her senaryo için arayüz ekran görüntüsü ve birim test sonucu gösterildi. Üç modül için üç birim test sınıfı yazılmış. İçlerinde toplam 38 test çalıştırılmış ve hepsi başarılı olmuş.

Sonuçta yazılımın, proje yönetim planı doğrultusunda gerçekleştirildiği ve gereksinimleri karşıladığının doğrulandığı söylendi.

Hoca, arkadaşlarına göstermesi için testleri canlı çalıştırmasını istedi. PHP ile yazılan testlerde gösterim şöyle ilerledi:

- Kategori test sınıfında önce bir kategori tanımlanıyor, sonra ekleme işlemi yapılıyor, sonra sonucun tanımlanan kategoriyle aynı olup olmadığı kontrol ediliyor. Bir testte beş doğrulama (assertion) var ve hepsi geçiyor.
- Beklenen ad “test2” olarak değiştirilince girilen ve beklenen değer farklı olduğu için test başarısız oluyor.
- Başka bir yerdeki kod bilerek bozulunca ilgili test yine başarısız oluyor. Sunan öğrenci bunun sık yaşanan bir duruma karşı faydalı olduğunu söyledi: Bir yeri değiştirirken başka bir yeri bozmak.
- Tüm ürün testleri bir anda da çalıştırılabiliyor.

Hoca, test derslerinde genellikle Java ve JUnit kullanıldığını, PHP’de birim test görmenin iyi olduğunu söyledi.

8. Şelale Modelinde İş Adımları Arasındaki İlişkiler

Dersin en önemli kavramsal uyarısı, 3. uygulamanın ProjectLibre planı gösterilirken geldi. Sunan öğrenci şelale modeli olduğu için görevlerin birbirine sırayla bağlandığını, birinin bitince diğerinin başladığını söylemişti. Hoca bunun bir yanılgı olduğunu belirtti.

> **Hocanın vurgusu:** Şelale modeli olunca bütün alt iş adımlarının bitiş-başlangıç (finish-to-start) gitmesi şart değildir. Bunu diğer arkadaşlarınız için de söylüyorum, bunu bir koşul gibi düşünmeyin.

Üst düzey aşamalarda ilişki gerçekten bitiş-başlangıçtır: Analiz bitince tasarım, tasarım bitince gerçekleme, gerçekleme bitince test başlar. Ama bir aşamanın içindeki alt işlemler için durum farklıdır. Alt işlemler aynı anda yürüyebilir. Hocanın örneği SRS dokümanının yazılmasıydı: Dokümanın bölümleri paralel gidebilir, ama aşamanın sonunda hepsi tamamlanmış olur. Bu yüzden bazı aktiviteler arasında başlangıç-başlangıç (start-to-start) ilişkisi de kurulabilir.

> **Ek bilgi (derste anlatılmadı):** Proje planlama araçlarında görevler arasında dört tür bağımlılık tanımlanır: bitiş-başlangıç, başlangıç-başlangıç, bitiş-bitiş ve başlangıç-bitiş. En sık kullanılan bitiş-başlangıçtır. Hocanın sözünü ettiği “birinin bitmesi için diğerinin de bitmesi gerekebilir” durumu bitiş-bitiş ilişkisine karşılık gelir.

Dersin Toparlaması

Bu derste aynı konuya, yani yazılım projesinin başarısına üç farklı açıdan bakıldı. 1. sunum, hataların kaynağında insan olduğunu ve hataları azaltmak için yalnız iradenin yetmediğini gösterdi. Süreç, araç ve yönetim desteği gerekiyor. 2. sunum, COCOMO II’nin güvenlik kritik bir yazılımda gerçekleşen eforun altında kaldığını gösterdi: 1257 kişi-ay tahmine karşılık 1780

kişi-ay. Doğrulama ve gözden geçirme ağırlığı yüksek süreçlerde model doğrudan kullanıldığında yetersiz kalıyor. 3. sunum ise Scrum'ın bilgi alanları üzerindeki olumlu etkisini anket sonuçlarıyla ele aldı.

Uygulama sunumlarında derste görülen belgelerin küçük projelerde nasıl hazırlandığı görüldü: risk tablosu, WBS, Gantt şeması, kalite ölçüleri, fonksiyon noktası ve COCOMO tahmini. Kavramsal olarak akılda kalması gereken nokta şudur: Şelale modeli üst düzey aşamaları sıraya koyar, ama alt iş adımları paralel de yürüyebilir. Uygulama sunumları sonraki derste devam edecek.

Not: İsterseniz bu kayıtları NotebookLM gibi bir araca atıp daha istifadeli hale getirebilirsiniz.